

CONCURRENCY AND SCHEME INTRO Meta

COMPUTER SCIENCE MENTORS 61A

November 4 – November 7, 2025

Example Timeline

- Ice Breaker [3 min]

If you want, start off with an icebreaker of your choice!

Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!

- Mini-lecture: Concurrency (This is a new topic, so see the lecture slides for more info!)
- Q1: Sleepy [5 min]
- Q2: Breakfast [8 min]
- Mini-Lecture: Scheme Intro
- Q3: What would Scheme do? [5 min]
- Q4: Hailstone [7 min]
- Q5: Apply N Times [10 min]

1 Concurrency

1. How many seconds would this function call take to run?

```
>>> async def sleepy(a):  
    await asyncio.sleep(a)  
    await asyncio.sleep(a-1)  
>>> async def sleepy2(b):  
    await asyncio.gather(sleepy(b), sleepy(b-1))  
>>> asyncio.run(sleepy2(5))
```

9

`asyncio.gather()` runs the arguments concurrently, so the total time this takes to run will just be the longest coroutine call.
`sleepy(5)` will take 5 + 4 seconds to run (since the `asyncio.sleep()` calls are **in** sequence/**not in** a gather call),
while `sleepy(4)` will take 4 + 3 seconds to run. So the total time taken will be 4 + 5 = 9 seconds.

(This **is** a new question! Please let us know **if** this answer seems wrong!)

Suggested Time: 5 min

2. Super Speedy Sam wants to make breakfast! It takes Sam two seconds to toast bread and one second to brew coffee. Both tasks should happen at the same time. Fill in the blanks for the functions below to make this happen.

```
import asyncio

async def make_task(name: str, delay: float):
    '''
    Simulates a breakfast task by waiting for a delay period of time.
    '''
    print(f'Starting: {name}')
    await asyncio.__(__)
    print(f'Finished: {name}')
    return f'{name} is ready'

async def main():
    '''
    Runs both breakfast tasks concurrently.

    >>> asyncio.run(main())
    Making breakfast...
    Starting: Toast
    Starting: Coffee
    Finished: Coffee
    Finished: Toast
    Breakfast is ready: ['Toast is ready', 'Coffee is ready']
    '''
    print('Making breakfast...')
    results = await asyncio.__(make_task(____, ____),
                               make_task(____, ____))
    print(f'Breakfast is ready: {results}')
```

```
import asyncio

async def make_task(name: str, delay: float):
    '''
    Simulates a breakfast task by waiting for a delay period of time.
    '''
    print(f'Starting: {name}')
    await asyncio.sleep(delay)
    print(f'Finished: {name}')
    return f'{name} is ready'

async def main():
    '''
    Runs both breakfast tasks concurrently.
```

```
>>> asyncio.run(main())
Making breakfast...
Starting: Toast
Starting: Coffee
Finished: Coffee
Finished: Toast
Breakfast is ready: ['Toast is ready', 'Coffee is ready']
'''
print('Making breakfast...')
results = await asyncio.gather(make_task('Toast', 2.0),
                               make_task('Coffee', 1.0))
print(f'Breakfast is ready: {results}')
```

For Fall 2025 mentors (if future sem feel free to delete this): PLEASE give us feedback on this section, since all of the concurrency questions are new and this is a new topic! Make sure to review what each async function does (like sleep and gather), and feel free to ask clarifying questions or probe more about why concurrency is important.

Suggested Time: 8 min

2 Scheme Intro

1. What will Scheme output?

```
scm> (define pi 3.14)
```

```
pi
```

- Terminal will give you back the name of whatever you just defined

```
scm> pi
```

```
3.14
```

- In this case, you will get the value that pi is assigned to

```
scm> 'pi
```

```
pi
```

- Scheme doesn't evaluate the next expression after the quote.

```
scm> (+ 1 2)
```

```
3
```

```
scm> (+ 1 (* 3 4))
```

```
13
```

- This tests the order of operations in Scheme.

```
scm> (if 2 3 4)
```

3

```
scm> (if 0 3 4)
```

3

- Unlike Python, all Scheme values other than `#f` are true. Therefore, 0 is a true value, and this expression evaluates to 3.

```
scm> (- 5 (if #f 3 4))
```

1

```
scm> (if (= 1 1) 'hello 'goodbye)
```

hello

- In this case, there's a similar behavior to Python
- Emphasize the importance of short circuiting – it will be very important for the Scheme project

```
scm> (define (factorial n)
      (if (= n 0)
          1
          (* n (factorial (- n 1)))))
```

factorial

```
scm> (factorial 5)
```

120

Teaching Tips

- We can compare this factorial implementation in Scheme to the Python interpretation so that the students can see how the `define` keyword works
- If your students are not caught up with lecture/have had no practice with Scheme, try drawing parallels between control statements in Scheme and Python.

Suggested Time: 5 min

2. Define a procedure called `hailstone`, which takes in two numbers `seed` and `n` and returns the `n`th number in the hailstone sequence starting at `seed`. Assume the hailstone sequence starting at `seed` has a length of at least `n`. As a reminder, to get the next number in the sequence, divide by 2 if the current number is even. Otherwise, multiply by 3 and add 1.

Useful procedures

- `quotient`: floor divides, much like `//` in python
`(quotient 103 10)` outputs 10
- `remainder`: takes two numbers and computes the remainder of dividing the first number by the second
`(remainder 103 10)` outputs 3

```
; The hailstone sequence starting at seed = 10 would be
; 10 => 5 => 16 => 8 => 4 => 2 => 1
```

```
; Doctests
> (hailstone 10 0)
10
> (hailstone 10 1)
5
> (hailstone 10 2)
16
> (hailstone 5 1)
16
```

```
(define (hailstone seed n)

  (if (_____)

      _____

      (if (_____) (_____)

          (_____

              (_____)

              (_____)

          )

          (_____

              (+ ____ (* _____))

              (_____)

          )

      )

  )

)
```

```
(define (hailstone seed n)
  (if (= n 0)
      seed
      (if (= 0 (remainder seed 2))
          (hailstone
            (quotient seed 2)
            (- n 1))
          (hailstone
            (+ 1 (* seed 3))
            (- n 1))))))
```

; Alternative solution **with** cond

```
(define (hailstone seed n)
  (cond
    ((= n 0) seed)
    ((= 0 (remainder seed 2))
     (hailstone
      (quotient seed 2)
      (- n 1)))
    (else
     (hailstone
      (+ 1 (* seed 3))
      (- n 1)))))
```

Students have seen hailstone before. The goal with this problem is to get the students comfortable with Scheme by having them solve a familiar problem in an unfamiliar language. However, students may find this to be boring because they have seen it before. If this is the case, you can feel free to skip this problem.

Teaching Tips

Python version:

```
def hailstone(seed, n):
    if n == 0:
        return seed
    if seed % 2 == 0:
        return hailstone(seed//2, n - 1)
    else:
        return hailstone(3*seed + 1, n - 1)
```

- If they're confused, point them towards the % in Python and how they can get the same value back in Scheme (answer: remainder function)
- Remind them to be careful about parentheses
- If you don't use cond, how might you write an equivalent function?
- In problems like this, I like to emphasize to my students how valuable it is to indent their code so that they can keep everything well organized. Scheme can be very confusing when not properly formatted because the language seems to just be an endless stream of parentheses.

Suggested Time: 7 min

3. Define **apply-multiple** which takes in a single argument function *f*, a nonnegative integer *n*, and a value *x* and returns the result of applying *f* to *x* a total of *n* times.

```
;doctests
scm> (apply-multiple (lambda (x) (* x x)) 3 2)
256
scm> (apply-multiple (lambda (x) (+ x 1)) 10 1)
11
scm> (apply-multiple (lambda (x) (* 1000 x)) 0 5)
5
```

```
(define (apply-multiple f n x)
```

```
_____
_____
_____
```

```
)
```

```
(define (apply-multiple f n x)
  (if (= n 0)
      x
      (f (apply-multiple f (- n 1) x)))))
```

Alternate solution:

```
(define (apply-multiple f n x)
  (if (= n 0)
      x
      (apply-multiple f (- n 1) (f x))))
```

Teaching Tips

- Functions can get a little confusing to work with in Scheme so it helps to remind your students that they are all just pieces of data.
- Thinking about the base case first may be more helpful with this problem. When do you stop applying your function to the input? How do you know/which input will tell you when to stop? This will then provide a good idea of what the recursive calls should be.
- There are two alternate solutions to this issue that differ in what you call your function *f* on. Go along with whatever your students share and if possible share the alternate solutions so students can see different ways of recursing in Scheme.

Suggested Time: 10 min